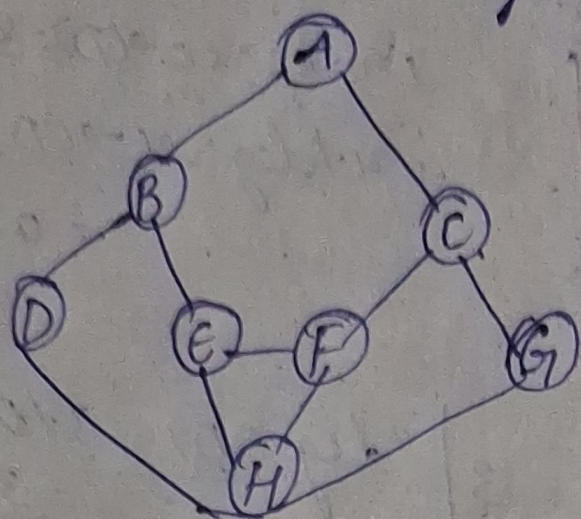
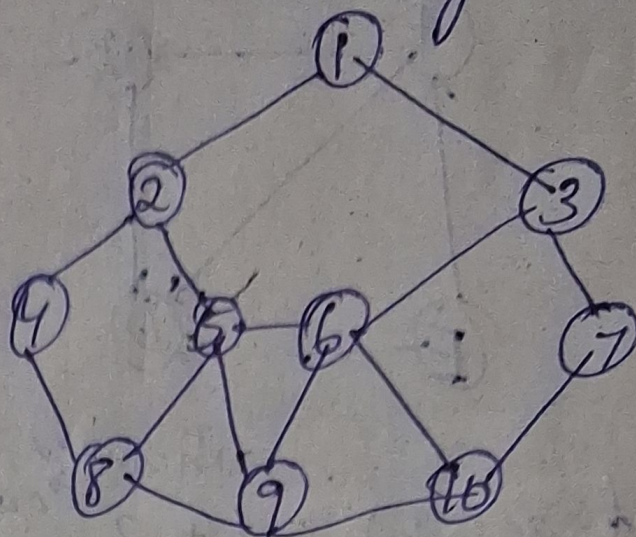


22/9 H/w

Graph-1 (Undirected, Unweighted)



Graph-2 (Undirected, Unweighted)

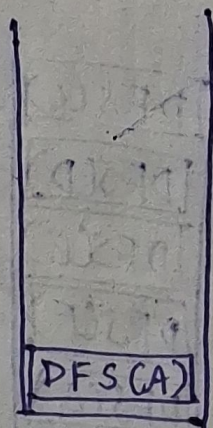


i) Procedure :-

i) Start with node A. DFS(A) is called. Initially, A is unvisited. so, update A as visited and output it.

Adjacency list:-

DFS order: A



Call stack

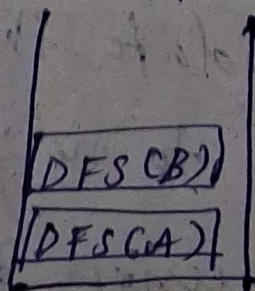
X $\emptyset$

A: [B, C]
B: [A, D, E]
C: [A, F, G]
D: [B, H]
E: [B, F, H]
F: [C, E, H]
G: [C, H]
H: [D, E, F, G]

ii) Now, Go to the first neighbour of A, i.e., B, check if B is unvisited. As it is, update it as visited & output it.

DFS order:

A B

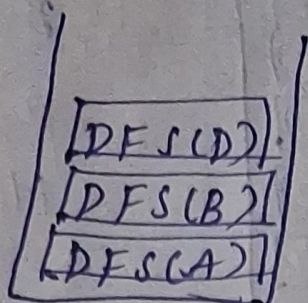


A: [X, C]
B: [X, D, E]

iii) Check the neighbours of B whether they are visited or unvisited. A is visited, then D is unvisited. So, update it as visited & o/p it.

DFS order:

A B D



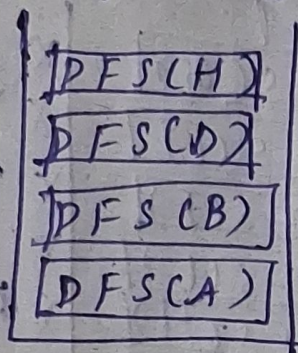
A: [~~X~~, C]

B: [~~X~~, ~~X~~, E]

iv) Now, check the neighbours of D. B is already visited. then, go to H. it's unvisited. update it as visited & o/p it.

DFS order:

A B D H



A: [~~X~~, C]

B: [~~X~~, ~~X~~, E]

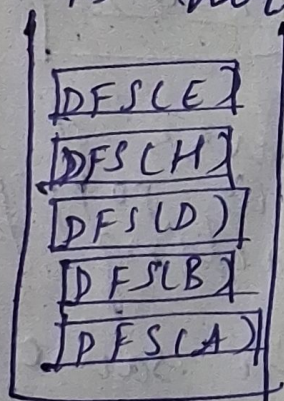
D: [~~X~~, ~~X~~]

H: [~~X~~, E, F, G]

v) Now, check the neighbours of H. D is visited. E is not. so, update it as visited & o/p it.

DFS order:

A B D H E



A: [~~X~~, C]

B: [~~X~~, ~~X~~, ~~X~~]

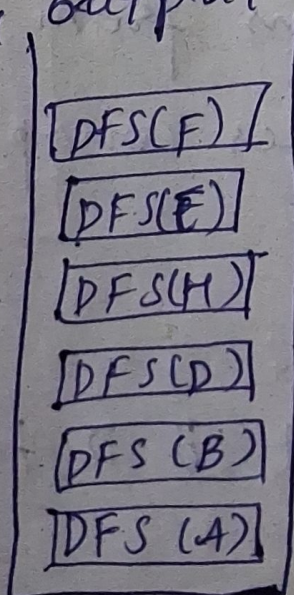
D: [~~X~~, ~~X~~]

H: [~~X~~, ~~X~~, F, G]

vi) Now, F is not visited, so update it as visited & output it.

DFS order:

A B D H E F



A: [~~X~~, C]

B: [~~X~~, ~~X~~, ~~X~~]

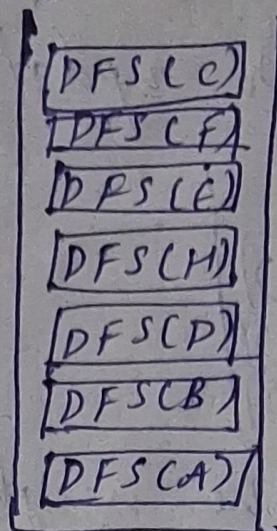
D: [~~X~~, ~~X~~]

H: [~~X~~, ~~X~~, ~~X~~, G]

vii) Now, check neighbours of F. C is not visited. update it as visited & o/p it.

DFS order:

A B D H E F C



A: [~~X~~, ~~X~~]

B: [~~X~~, ~~X~~, ~~X~~]

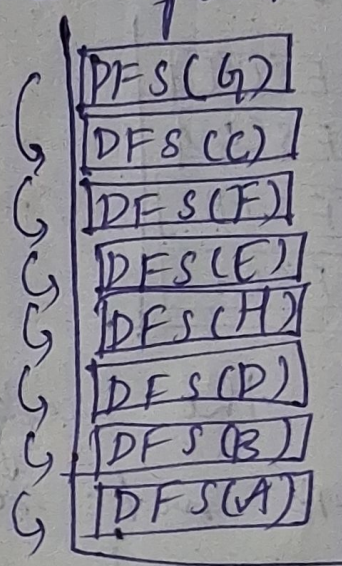
D: [~~X~~, ~~X~~]

H: [~~X~~, ~~X~~, ~~X~~, G]

viii) Now, check neighbours of C. A, F are visited. G is unvisited. update it as visited & o/p it.

DFS order:

A B D H E F C G



~~X~~ A: [~~X~~, ~~X~~]

~~X~~ B: [~~X~~, ~~X~~, ~~X~~]

~~X~~ D: [~~X~~, ~~X~~]

~~X~~ H: [~~X~~, ~~X~~, ~~X~~, G]

~~X~~

~~X~~

ix) Check the neighbours of G. All are visited. Hence, backtracking taking place.

Now, DFS(C). Again same, so, since all nodes are visited, backtracking comes to

DFS(A) and returns from the function.

So, call stack becomes completely empty.

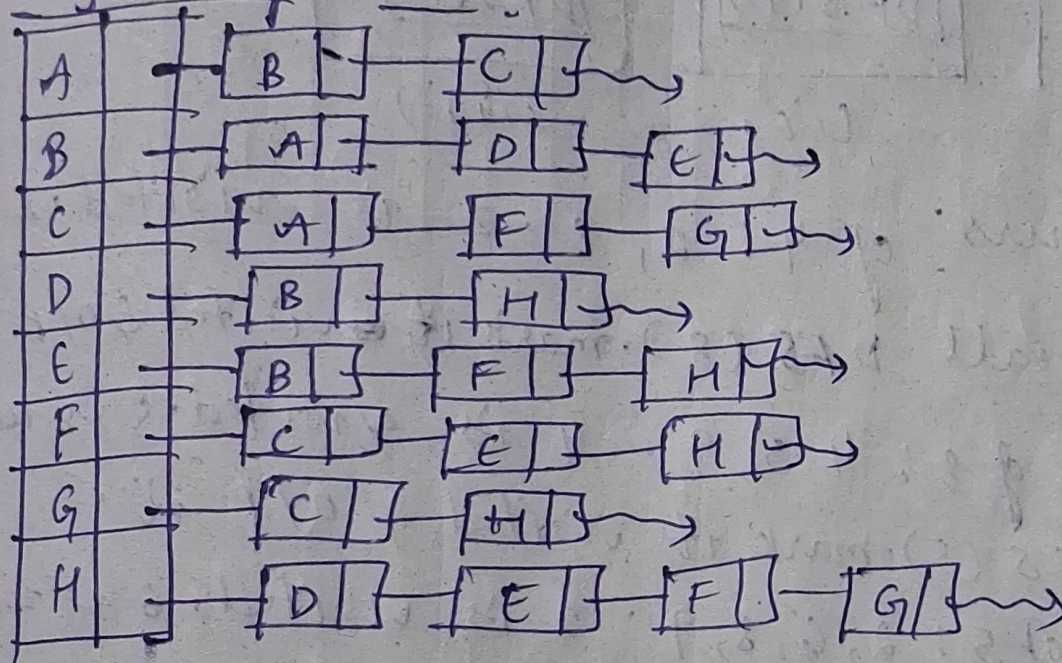
([])

Final DFS order: A B D H E F C G.

Adjacency Matrix :-

	A	B	C	D	E	F	G	H
A		1	1					
B	1			1	1			
C	1					1	1	
D		1						1
E		1				1		1
F			1		1			1
G			1					1
H				1	1	1	1	

Adjacency List :-



2) Procedure :-

Adjacency List :-

1 : [~~1~~, ~~2~~] 10 : [~~6~~, ~~7~~, ~~9~~]

2 : [~~1~~, ~~2~~, ~~3~~]

3 : [~~1~~, ~~2~~, ~~3~~]

4 : [~~2~~, ~~8~~]

5 : [~~2~~, ~~8~~, ~~9~~]

6 : [~~3~~, ~~5~~, ~~9~~, ~~10~~]

7 : [~~8~~, ~~10~~]

8 : [~~1~~, ~~5~~, ~~9~~]

9 : [~~5~~, ~~6~~, ~~8~~, ~~10~~]

DFS order :-

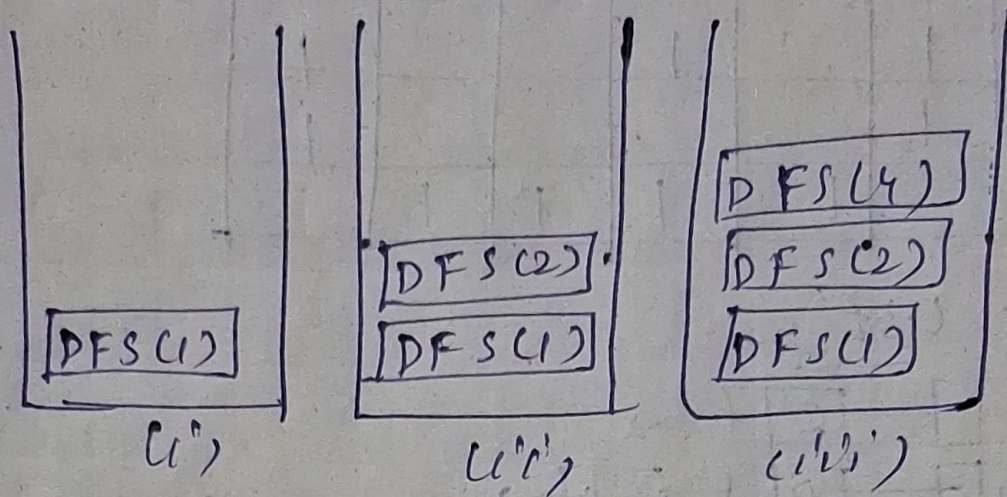
1 2 4 8 5 6 3

7 10 9

i) Start with Node - 1, DFS(1). Mark it as visited & add it to DFS order.

ii) Now, neighbours of 1 are 2 & 3. 2 is unvisited, call DFS(2). mark it as visited & add it to DFS order.

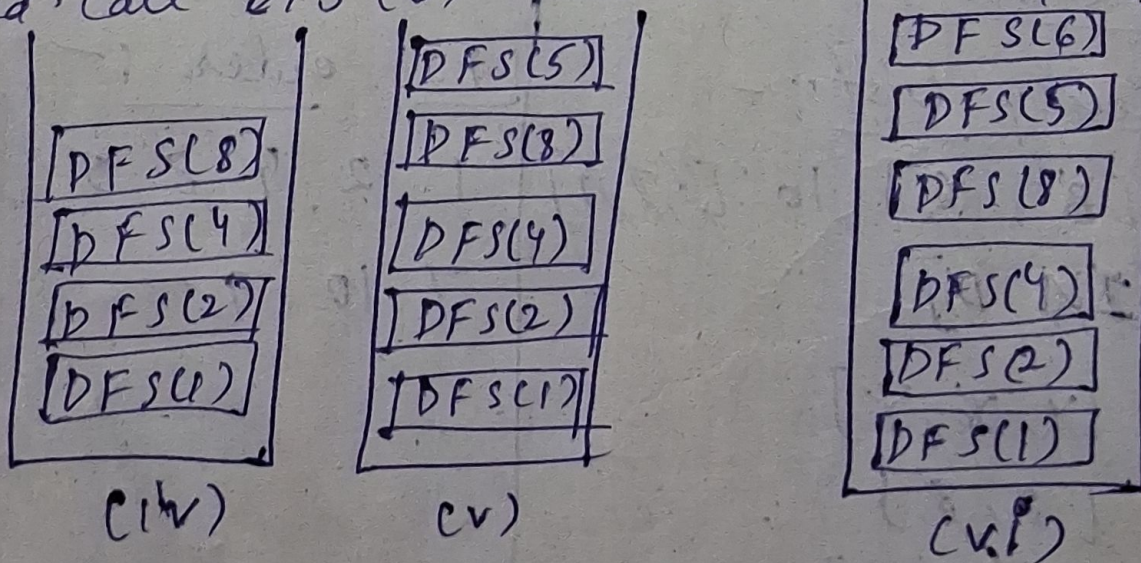
iii) Neighbours of 2: 1, 4, 5. 1 is visited, 4 is unvisited. call DFS(4), mark it as visited & add it to DFS order.



iv) Neighbours of 4: 2, 8. 2 is visited, 8 is unvisited. call DFS(8). mark it as visited & o/p it.

v) Neighbours of 8: 4, 5, 9. 4 is visited, 5 is unvisited. call DFS(5). mark it as visited & o/p it.

vi) Neighbours of 5: 2, 6, 8, 9. 2 is visited, 6 is unvisited. call DFS(6). mark it as visited & o/p it.

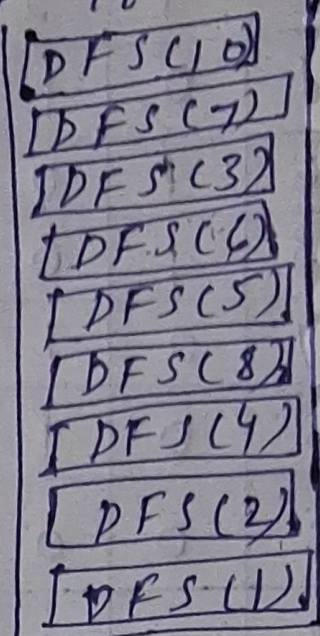
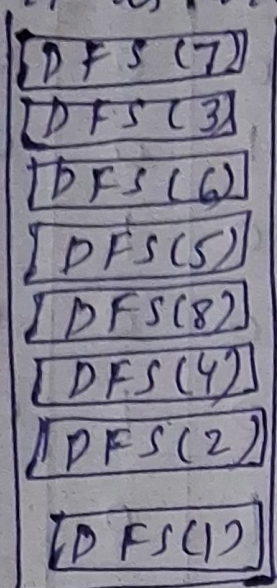
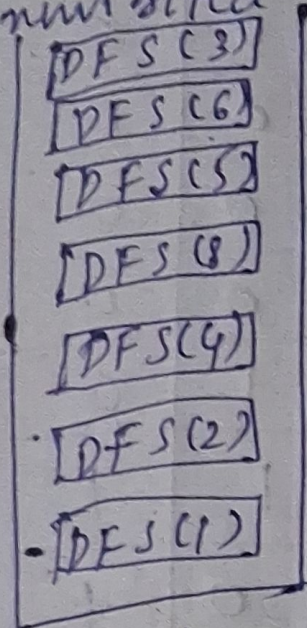


vii) Neighbours of 6: 3, 5, 9, 10. 5 is visited, 3 is unvisited. call DFS(3). mark it as visited & o/p it.

viii) Neighbours of 3: 1, 6, 7. 1 & 6 are visited, 7 is unvisited. call DFS(7). mark it as visited & o/p it.

o/p it.

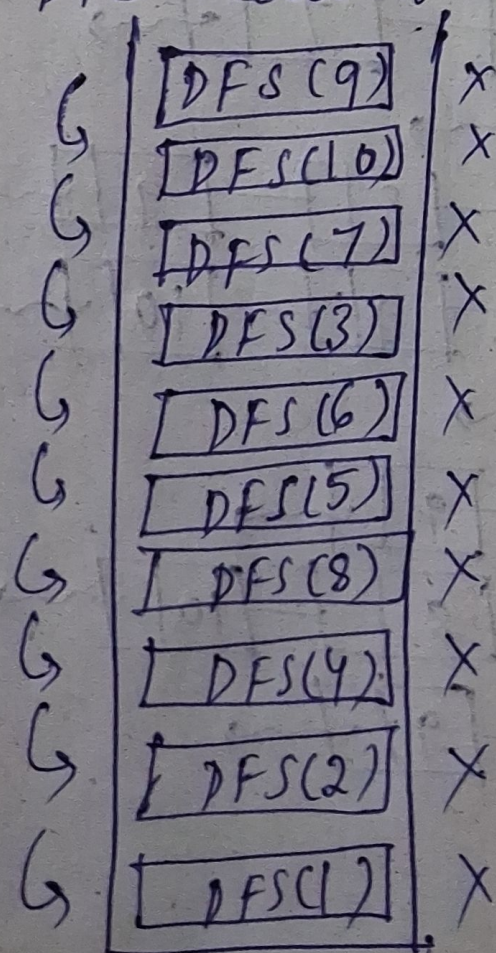
ix) neighbours of 7: 3, 10, 3 is visited. 10 is unvisited. mark it as visited & o/p it.



x) Neighbours of 10: 6, 7, 9. 6, 7 are visited. 9 is unvisited. mark it as visited & o/p it.

xi) Neighbours of 9: 5, 6, 8, 10. All are visited. Hence, DFS(9) returns & popped. This goes on till — — — DFS(1) returns & popped. Now, call stack becomes empty.

∴ final DFS order: 1 2 4 8 5 6 3 7 10 9.



(X) & (X)

Adjacency Matrix :-

	1	2	3	4	5	6	7	8	9	10
1	0	1	1	0	0	0	0	0	0	0
2	1	0	0	1	1	0	0	0	0	0
3	1	0	0	0	0	1	1	0	0	0
4	0	1	0	0	0	0	0	1	0	0
5	0	1	0	0	0	1	0	1	1	0
6	0	0	1	0	1	0	0	0	1	1
7	0	0	1	0	0	0	0	0	0	1
8	0	0	0	1	1	0	0	0	1	0
9	0	0	0	0	1	1	0	1	0	1
10	0	0	0	0	0	1	1	0	1	0

Adjacency List :-

